```
X::Y.local_constants # => [:Y1, :X1]
```
The names are returned as symbols.

Defined in `active_support/core_ext/module/introspection.rb`.

3.4.1 Qualified Constant Names

The standard methods `const_defined?`, `const_get`, and `const_set` accept bare constant names. Active Support extends this API to be able to pass relative qualified constant names.
The new methods are `qualified_const_defined?`, `qualified_const_get`, and `qualified_const_set`. Their arguments are assumed to be qualified constant names relative to their receiver:
```
Object.qualified_const_defined?("Math::PI")       # => true
Object.qualified_const_get("Math::PI")            # =>
3.141592653589793
Object.qualified_const_set("Math::Phi", 1.618034) # => 1.618034
```
Arguments may be bare constant names:

```
Math.qualified_const_get("E") # => 2.718281828459045
```
These methods are analogous to their built-in counterparts. In particular, `qualified_constant_defined?` accepts an optional second argument to be able to say whether you want the predicate to look in the ancestors. This flag is taken into account for each constant in the expression while walking down the path.
For example, given

```
module M
  X = 1
end

module N
  class C
    include M
  end
end
```
`qualified_const_defined?` behaves this way:
```
N.qualified_const_defined?("C::X", false) # => false
N.qualified_const_defined?("C::X", true)  # => true
N.qualified_const_defined?("C::X")        # => true
```
As the last example implies, the second argument defaults to true, as in `const_defined?`.
For coherence with the built-in methods only relative paths are accepted. Absolute qualified constant names like `::Math::PI` raise `NameError`.
Defined in `active_support/core_ext/module/qualified_const.rb`.

3.5 Reachable

A named module is reachable if it is stored in its corresponding constant. It means you can reach the module object via the constant.

That is what ordinarily happens, if a module is called "M", the `M` constant exists and holds it:
```
module M
end

M.reachable? # => true
```
But since constants and modules are indeed kind of decoupled, module objects can become unreachable: